

PageContext: Context Management for Long-Horizon Agentic Systems

Yunqi Li

yunqili4@illinois.edu

Yuchen Wang

yuchen87@illinois.edu

Abstract

Long-horizon agentic systems accumulate large amounts of task context, including user queries, intermediate results, tool outputs, and error logs. However, limited context windows make it difficult for agents to retain all relevant information across extended interactions. Existing approaches such as summarization, sliding windows, and retrieval-augmented memory either risk losing fine-grained details or depend heavily on retrieval quality. Inspired by cache management in operating systems, we propose PageContext, a hybrid memory management framework for long-horizon agents. PageContext is built on the FREQUENCY LOCALITY hypothesis, which posits that certain pieces of information are repeatedly useful across many queries. The system maintains frequently accessed pages in a persistent cache governed by a Least Frequently Used (LFU) policy, while still retrieving query-specific pages from a vector database at inference time. We evaluate PageContext on the *LoCoMo* benchmark across eight open-source and commercial LLM backends. The results show that PageContext performs on par with strong retrieval-only baselines: persistent pages help when they carry truly task-wide information, but can also introduce noise that slightly reduces F1 when the persistent slots are filled by merely-popular content. Our analysis highlights the importance of adaptive memory policies, heterogeneous retrieval strategies, and multi-granularity context organization for future long-horizon agentic systems.

1 Introduction

With the rapid development of large language models (LLMs) [2, 6], the dominant research paradigm has shifted from training models to merely generate text toward training them to *interact* with external environments in order to accomplish complex tasks. Such models are referred to as **agents**, and they typically interact with their environment through **tools**; for example, a coding agent’s tools are usually shell commands. When an agent executes a task, the user query, tool outputs, error logs, intermediate reasoning, and additional user prompts are concatenated together as the model’s input by default. However, LLMs can only condition on a bounded amount of information, called their **memory**¹ (a.k.a. context window). Once the accumulated input exceeds this budget, the model is no longer guaranteed to attend to all of the supplied information. We refer to tasks whose context naturally outgrows the model’s context window as **long-horizon** tasks.

Managing agent memory in long-horizon tasks is a long-standing question. The most straightforward solution is to *summarize* the

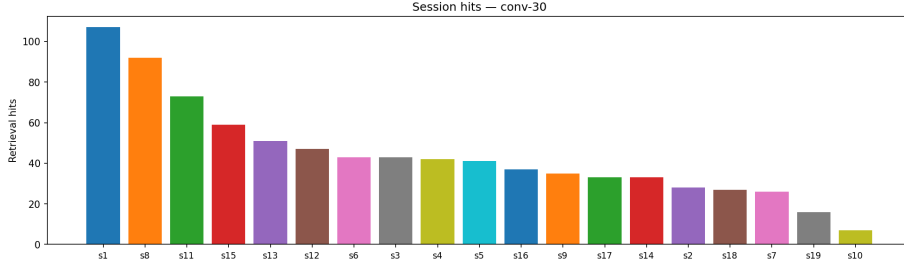
accumulated context whenever it exceeds the context window, an approach adopted by state-of-the-art coding agents such as Claude Code. Summarization, however, inevitably discards fine-grained details and is therefore most suitable for high-volume, disposable content such as tool execution logs. A second common practice is to maintain a *sliding window* whose length matches the context-window capacity, advancing along the timeline of the agent’s interaction and dropping outdated context. While effective for models with very long context windows, this is problematic for smaller models, whose limited windows cause them to lose track of earlier information. A third popular practice is to store all past information as semantic embeddings in a vector database and, for each new query, retrieve the most relevant entries to feed back to the agent as memory. This is essentially retrieval-augmented generation (RAG), and its quality depends heavily on the embedding model and the retrieval similarity function.

An analogous problem arises in operating systems (OS). Modern systems use cache mechanisms to reduce the cost of accessing slow storage or memory layers: instead of repeatedly fetching data from a slow device, the OS keeps recently accessed data (e.g., memory pages) in a faster cache. Because cache space is limited, the system must decide which items to keep and which to evict when new data is loaded. A common principle guiding this decision is **temporal locality**—data accessed recently is likely to be accessed again in the near future. Building on this observation, the Least Recently Used (LRU) policy organizes cached items by their most recent access time: an access marks an item as recently used, and when the cache is full the item not accessed for the longest time is evicted. LRU thus uses recent access history as a simple proxy for future utility.

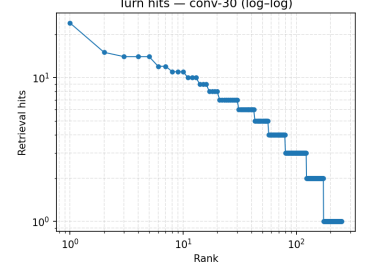
Inspired by OS cache design, we propose a context management scheme PageContext for agentic systems on long-horizon tasks. PageContext employs a hybrid paradigm: it maintains a small set of hot-accessed pages persistently across the task (analogous to an OS cache), and at the same time uses similarity search to retrieve relevant information from the full content store at each query (analogous to main memory). To summarize, we make the following contributions:

- We propose the FREQUENCY LOCALITY hypothesis, an analogue of temporal locality that motivates the design of PageContext (Section 2).
- We formalize PageContext as a hybrid two-tier system for managing agent memory in long-horizon tasks (Section 3).
- We conduct a comprehensive evaluation of PageContext on eight LLM backends—both open-source and commercial—and analyze the factors that drive its performance (Section 4).

¹As of 2026, commercial LLMs such as ChatGPT-5.5 and Claude-Opus-4.7 accept up to 1M tokens of memory, while smaller open-source LLMs have much smaller context windows (e.g., 10K tokens).



(a) Distribution of session frequency in retrieval results.



(b) Rank–turn frequency in retrieval results.

Figure 1: Heavy-tailed retrieval frequency on *LoCoMo* conversation #30, supporting Hypothesis 1. Similar patterns hold for the remaining nine conversations.

2 Preliminary

By analogy with temporal locality, we define the frequency locality hypothesis:

HYPOTHESIS 1 (FREQUENCY LOCALITY). *In long-horizon tasks, certain pieces of content are universally relevant across many queries.*

To test this hypothesis we use the *LoCoMo* dataset [5], a pioneering benchmark for the long-horizon conversational memory of LLM agents. The dataset contains 10 long conversations, with each conversation comprising 27 sessions on average and each session containing about 21 turns. Each conversation involves two speakers, so a single turn captures the complete utterance of one speaker. The notion of “content” in Hypothesis 1 is an atomic piece of information, and we therefore study the hypothesis at two granularities—*turn* and *session*.

We instantiate the retrieval-only memory described in Section 1. Each turn is encoded into a dense embedding using a sentence-transformer model² and stored in an in-memory vector index. For each question we retrieve the top-8 most relevant turns under *cosine* similarity, and record both the retrieved turn IDs and their parent sessions. Figure 1 reports the resulting retrieval-hit frequencies for conversation #30 of *LoCoMo*. At both granularities the distribution is heavy-tailed, which is exactly what frequency locality predicts. In Figure 1a, sessions *s1* and *s8* are retrieved far more often than any other session; in Figure 1b, the top-10 turns are each retrieved more than 10 times, while the vast majority of turns are retrieved only a handful of times—or never. The other nine conversations exhibit qualitatively similar patterns.

These hot-accessed sessions and turns therefore appear to carry conversation-wide information, such as recurring topics or salient facts about the speakers. This motivates our design choice: keeping such high-level information persistently in the agent’s memory across the entire task should help the agent answer a broader range of user queries.

3 Methodology

We now elaborate on the design of PageContext, summarized in Figure 2. Let A be an agent with effective memory budget M , interacting with a user U through a sequence of queries $Q = \{q_1, \dots, q_n\}$. Each query carries a timestamp t_i , with $t_i < t_j$ whenever $i < j$. Following Hypothesis 1, we treat each atomic piece of information as a *page* p_i . An external vector database $\mathcal{D}$ stores the semantic embeddings of all observed pages $\mathcal{P} = \{p_i\}$. At each query, the set of pages loaded into the model’s context must satisfy

$$\sum_{p_i \in M} |p_i| \leq |M|,$$

so that the input fits within the agent’s context window.

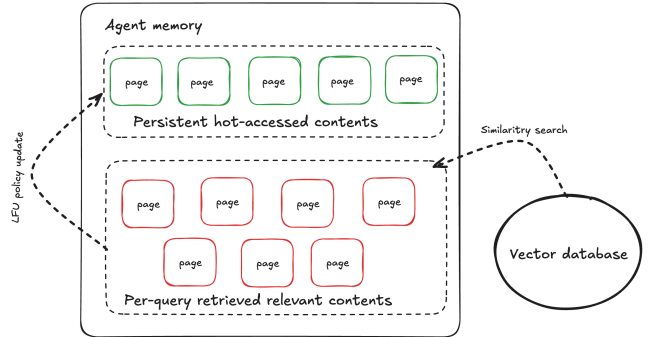


Figure 2: The PageContext framework. Green pages are persistent (hot) contents kept across the task, while red pages are retrieved per query. After the agent answers each query, the persistent set is updated via an LFU policy.

3.1 PageContext Framework Design

Building upon the frequency locality hypothesis, PageContext treats agent memory as a dynamic, multi-tiered system, reminiscent of modern hierarchical storage in operating systems. The memory hierarchy consists of:

- **Persistent cache (hot pages):** Maintains a set of frequently accessed atomic information units across the task. These hot

²<https://huggingface.co/BAAI/bge-small-en-v1.5>

Algorithm 1 PageContext

Require: Agent A , vector database $\mathcal{D}$, queries Q , hot-cache size n_1 , retrieval size n_2 .

Ensure: Answers $\mathcal{A}$ to Q .

```

1:  $\mathcal{A} \leftarrow \emptyset$ ;  $\mathcal{P}_2 \leftarrow \emptyset$ ; frequency counter  $\mathcal{H} \leftarrow 0$ 
2: for  $q_i \in Q$  do
3:    $S \leftarrow \text{Score}(\mathcal{D}, q_i)$   $\triangleright$  cosine similarity for all pages
4:    $\mathcal{T} \leftarrow \text{TopK}(S, n_1 + n_2)$ 
5:    $\mathcal{H}[p] += 1$  for each  $p \in \mathcal{T}$   $\triangleright$  frequency update
6:    $\mathcal{P}_1 \leftarrow \text{TopK}(S \setminus \mathcal{P}_2, n_2)$   $\triangleright$  cold retrieval
7:    $M \leftarrow \mathcal{P}_2 \cup \mathcal{P}_1$ 
8:    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\text{Answer}(A, q_i, M)\}$ 
9:    $\mathcal{P}_2 \leftarrow \text{UpdateLFU}(\mathcal{P}_2, \mathcal{P}_1, \mathcal{H}, n_1)$ 
10: end for
11: return  $\mathcal{A}$ 

```

pages are updated via a Least Frequently Used (LFU) policy, ensuring that high-utility content remains in memory without overwhelming the agent's context window.

- **Retrieval layer (cold main memory):** Retrieves semantically relevant pages from an external vector database for each query, capturing transient or query-specific context that is not stored in persistent memory.

This hybrid design allows PageContext to balance *memory efficiency* and *context fidelity*, providing the agent with both globally important knowledge and locally relevant details. By leveraging semantic embeddings and similarity search, the retrieval layer ensures the agent's memory is *query-aware*, while the persistent cache enforces a notion of *task-wide continuity*.

Algorithm 1 formalizes the workflow shown in Figure 2. PageContext exposes two hyper-parameters: n_1 , the size of the persistent hot cache, and n_2 , the number of pages retrieved per query. By analogy with hardware caching, we expect $n_1 < n_2$. For each query q_i , the system (i) scores all pages in $\mathcal{D}$ against q_i (line 3); (ii) updates the global frequency counter for the top- (n_1+n_2) most relevant pages, regardless of whether they are currently in the hot cache (lines 4–5); (iii) retrieves the top- n_2 pages outside the current hot cache as the cold set $\mathcal{P}_1$ (line 6); (iv) assembles the agent's context as $M = \mathcal{P}_2 \cup \mathcal{P}_1$ and obtains an answer (lines 7–8); and finally (v) refreshes the hot cache $\mathcal{P}_2$ via the LFU policy described in Algorithm 2 (line 9).

3.2 Memory Management Policies

Persistent memory update. After each query, PageContext refreshes the persistent cache using the LFU policy in Algorithm 2. The frequency counter $\mathcal{H}$ has already been incremented during retrieval (line 5 of Algorithm 1), so the update simply admits each newly retrieved cold page $x \in \mathcal{P}_1$ into the hot cache M_{cache} and, whenever $|M_{\text{cache}}|$ exceeds the capacity n_1 , evicts the resident page with the lowest frequency. Because $\mathcal{H}$ accumulates across the entire task, freshly admitted pages with low historical frequency are typically the first to be evicted, while pages that have repeatedly appeared in top- (n_1+n_2) for past queries persist.

Algorithm 2 UpdateLFU: LFU-based cache update

Require: Current hot cache M_{cache} , newly retrieved pages $\mathcal{P}_1$, frequency counter $\mathcal{H}$, capacity C .

Ensure: Updated hot cache M'_{cache} .

```

1: for  $x \in \mathcal{P}_1$  do
2:   if  $x \notin M_{\text{cache}}$  then
3:     Insert  $x$  into  $M_{\text{cache}}$ 
4:   end if
5:   while  $|M_{\text{cache}}| > C$  do
6:      $x_{\text{evict}} \leftarrow \arg \min_{m \in M_{\text{cache}}} \mathcal{H}[m]$ 
7:     if  $\mathcal{H}[x_{\text{evict}}] < \mathcal{H}[x]$  then
8:       Remove  $x_{\text{evict}}$  from  $M_{\text{cache}}$ 
9:     end if
10:  end while
11: end for
12: return  $M'_{\text{cache}} \leftarrow M_{\text{cache}}$ 

```

This policy preferentially retains information that is useful across many queries, under the assumption that high frequency correlates with task-wide relevance. Compared with sliding-window approaches, LFU provides a principled mechanism to prioritize truly valuable content over content that merely happened to be produced recently.

Classical OS caches typically use Least Recently Used (LRU) or First-In-First-Out (FIFO) replacement, since temporal locality emphasizes the *recency* of accesses. In our frequency-locality setting, however, we instead care about the *number* of occurrences across the entire task. A page that has been heavily accessed in the past may not appear in the most recent C queries, yet it should still be retained because of its global utility. Conversely, a page that is frequent only within a small recent window may indicate a transient topic rather than task-wide knowledge, and LFU correctly de-prioritizes it. Importantly, evicting a page does not destroy its access history in $\mathcal{H}$: an evicted page can be re-admitted later, giving the policy a form of adaptivity reminiscent of LRU.

Query-time retrieval. For each incoming query q_i , the system performs a cosine similarity search over $\mathcal{D}$ and selects the top- n_2 pages that are *not* currently in $\mathcal{P}_2$. This avoids duplicating already-hot content in the context and lets the cold tier focus on query-specific information. The agent's effective context for query q_i is then

$$M_i = \mathcal{P}_2 \cup \mathcal{P}_1.$$

By separating task-wide and on-demand content, PageContext mitigates the limitations of fixed-length context windows while still exposing the agent to globally important knowledge.

3.3 Parameterization and System Hyperparameters

As described in Section 3.1, PageContext has two primary hyper-parameters: n_1 , the size of the persistent cache, and n_2 , the number of pages retrieved per query. The choice of n_1 controls how much of the budget is reserved for globally important content, while n_2 controls the breadth of query-specific information. Both can be tuned based on the agent's context-window capacity and the

complexity of the task, making the framework applicable to both small open-source LLMs and large commercial models.

4 Evaluation

We now present an in-depth evaluation of PageContext. PageContext is open-sourced at <https://github.com/EricLYunqi/CS510-Spring2026>.

4.1 Setup

Benchmark. We use the *LoCoMo* benchmark, the same dataset as in the preliminary study (Section 2). It contains 10 conversations and 1,986 question-answer pairs spanning five categories: multi-hop, single-hop, temporal, open-domain, and adversarial. We follow the official *LoCoMo* scoring protocol: predictions and gold answers are lowercased, stripped of punctuation and a small set of stop words, tokenized, and stemmed with the Porter stemmer; the per-question metric is the F1 of stemmed-token overlap. For multi-hop questions (category 1) we use multi-answer F1 (splitting both prediction and gold on commas and averaging the per-gold maximum); for temporal questions (category 3) only the first “;”-separated gold answer is graded; for adversarial questions (category 5) we score 1 if the prediction contains “no information available” or “not mentioned” and 0 otherwise. The reported number is the macro mean over all questions.

Baseline. The retrieval-only baseline is the system described in Section 2: each turn is encoded with BAAI/bge-small-en-v1.5 and, for each question, the top- k turns under cosine similarity are concatenated as the agent’s context. We consider two settings, Baseline-18 ($k = 18$) and Baseline-14 ($k = 14$), to isolate the effect of context size from the effect of adding persistent pages.

LLM backends. We evaluate on eight LLM backends. Four are open-source models from the Qwen3 family [1] (0.6B, 1.7B, 4B, and 8B), and four are commercial models: ChatGPT-5-mini [7], Claude-Haiku-4.5, DeepSeek-V4-Flash [3], and Kimi-K2.5 [9]. All commercial models are invoked via Microsoft Azure.

Testbed. Open-source models run on a single NVIDIA A100 GPU on Google Colab; commercial models are invoked from a MacBook Pro running macOS 15 with 24 GB of RAM.

Parameters. We cap generation at $\text{max_tokens} = 256$ for every query so the full sweep completes in a reasonable budget. For PageContext we use $n_1 = 4$ persistent pages and $n_2 = 14$ retrieved pages, so that PageContext and Baseline-18 feed the same total number of turns ($n_1 + n_2 = 18$) into the agent. This matched-budget comparison isolates the contribution of the persistent cache itself.

Our evaluation targets two research questions:

- **RQ1.** Does PageContext improve answer F1 over the retrieval-only baselines?
- **RQ2.** What drives the observed differences, and what does this imply for the design of agent memory?

4.2 Research Question 1

The main results are shown in Figure 3. First, the absolute F1 of all eight backends is below 20%. We attribute this to two factors:

(i) the max_tokens budget of 256 is shared between reasoning and final answer, and when the model spends most of it on reasoning the answer itself can be truncated; (ii) retrieving at most 18 turns is not enough coverage for many multi-hop questions in *LoCoMo*. As expected, commercial LLMs outperform open-source ones at every size.

Focusing on the relative comparison between methods, we observe two findings:

- PageContext performs on par with the two retrieval-only baselines—and is in fact *slightly worse* than them on most backends.
- Baseline-14 is slightly better than Baseline-18, suggesting that feeding fewer (but higher-quality) turns helps the agent answer better.

We defer a detailed analysis of the first finding to Section 4.3. The second finding points to a limitation of the underlying retrieval strategy. With only 18 retrieved turns per query, the retrieved context is often insufficient for hard questions; for the remaining questions, the retrieved turns—all selected by a single embedder under a single similarity function—tend to be semantically homogeneous and rarely contain an exact reference to the answer. Feeding more such homogeneous content does not provide additional evidence; instead, the tail of the ranking (turns ranked below 14, which are only marginally relevant) acts as distractor noise that degrades inference. The remedy is unlikely to be a single “sweet spot” for k ; rather, the retrieval layer should combine multiple complementary signals—e.g., semantic search to surface contextual information together with exact-match or BM25 lookup to surface specific tokens close to the answer. Moreover, longer contexts increase the risk of hallucinated tokens, which is particularly punishing under a token-overlap F1: spurious words inflate the prediction set and crush precision.

4.3 Research Question 2

We now examine why PageContext is slightly behind the retrieval-only baselines. A first explanation mirrors the second finding above: the 4 persistent pages are always included in the context, but they are not necessarily relevant to the current query and can therefore act as noise. Even the most frequently accessed sessions and turns in Section 2 are retrieved on the order of ~ 100 times across the whole conversation, which is enough to qualify as “hot” but is far from making them universally on-topic. In other words, our hot pages do capture high-level information, but that is rarely the kind of exact reference a question grader rewards on *LoCoMo*.

To make this concrete, we break down per-question F1 scores for the four commercial LLMs (which obtain the highest absolute scores). As shown in Figure 4, the per-question F1 distribution is highly skewed: with the exception of DeepSeek-V4-Flash, the bulk of answers from the other three commercial models score below 0.4. This is also why DeepSeek-V4-Flash takes the lead in Figure 3. More importantly, the three methods’ distributions are nearly indistinguishable for scores greater than zero; the gap is concentrated in the *zero-score mass*, where PageContext produces slightly more zero-score answers than the two baselines. This is the dominant reason why PageContext trails the baselines on aggregate F1.

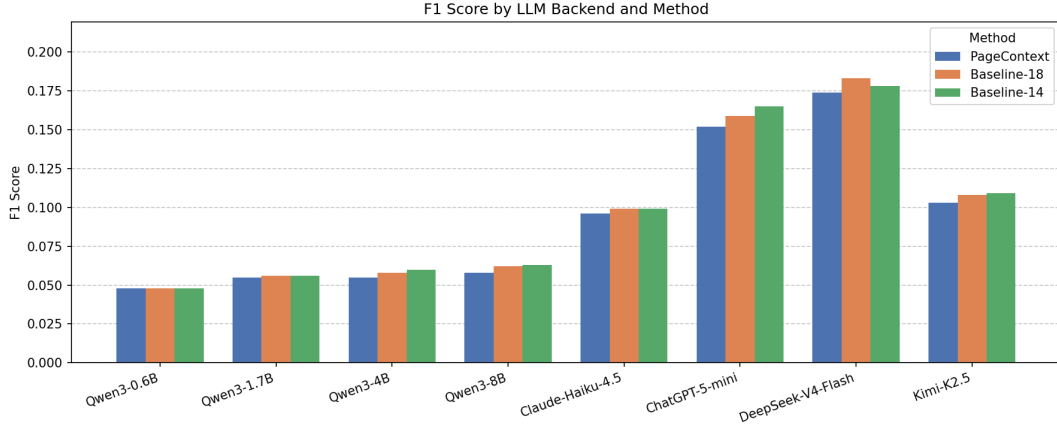


Figure 3: F1 scores of Baseline-18, Baseline-14, and PageContext on *LoCoMo* across four open-source (Qwen3 0.6B/1.7B/4B/8B) and four commercial LLM backends.

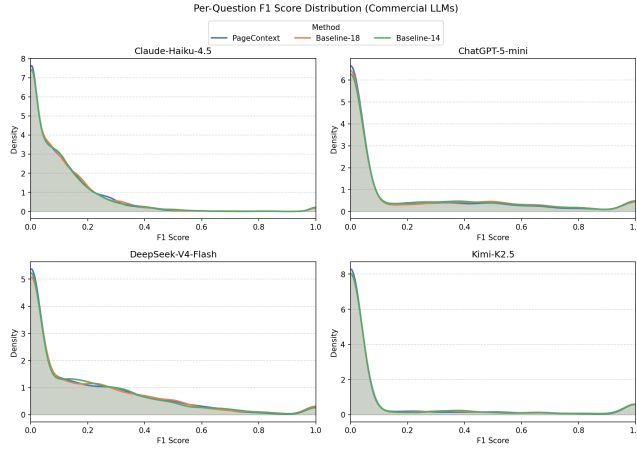


Figure 4: Per-question F1 distribution for the four commercial LLMs. The three methods are nearly indistinguishable for non-zero scores; the gap comes from the zero-score mass.

To investigate why so many answers score exactly zero, we compare the lengths of model outputs to gold answers. Ideally the two should be comparable: *LoCoMo* answers are typically only a few tokens long. Figure 5 shows this comparison for the commercial LLMs; for ChatGPT-5-mini and Kimi-K2.5 we exclude empty answers.³ Across the board, model answers are substantially longer than the gold. Given that the retrieved context is semantically homogeneous and rarely contains the exact reference, the models tend to hedge with auxiliary words and reasoning fragments. This drives recall up modestly but precision sharply down, which under stemmed-token F1 produces a long tail of zero-score outputs.

³Roughly 53% of ChatGPT-5-mini outputs and 73% of Kimi-K2.5 outputs are empty. We believe this is because either the model declined to answer, or the response was filtered by Azure’s content policy.

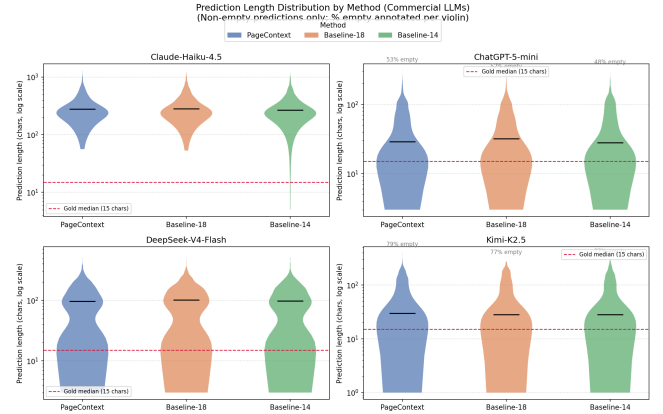


Figure 5: Length distribution of model answers vs. gold answers on *LoCoMo* for the four commercial LLMs. Empty answers are excluded for ChatGPT-5-mini and Kimi-K2.5.

5 Discussion

5.1 Unsolved Challenges

Several challenges remain open. First, LFU treats page importance in a fairly static way and does not model evolving task priorities or dependencies between pieces of content. Second, the split between persistent and retrieval-based context is currently fixed via n_1 and n_2 , which limits adaptivity in tasks whose information requirements vary substantially across time. Finally, our *LoCoMo*-based study only exercises conversational pages of a single granularity (turns) and is therefore conservative; long-horizon agentic tasks in the wild also contain disposable artefacts (tool logs, scratch reasoning) that should not be treated as pages at all.

5.2 Future Directions

We see two complementary axes for future work, which together form an “algorithm–system co-design” agenda.

Algorithm perspective. PageContext would benefit from: (1) *Heterogeneous retrieval*. Combining multiple embedders and similarity functions (e.g., dense semantic search, BM25, exact-match) at retrieval time would diversify the candidate set fed into the LFU update, leading to a more genuinely high-utility persistent cache. (2) *Adaptive page chunking*. The current evaluation conveniently inherits *LoCoMo*'s turn/session structure, but in general the granularity of a page is a design choice with real consequences: turns are too fine to be globally informative, sessions are too coarse to fit in the budget. An adaptive scheme could start with fine-grained chunks when the task is short and re-chunk into coarser units as the task grows, rebuilding $\mathcal{D}$ and $\mathcal{M}$ on the fly. (3) *Page-vs.-junk classification*. Real agentic tasks contain disposable content—tool logs, retried commands, error traces—that should be filtered out before reaching the page store.

System perspective. A promising direction is *adaptive memory policies* that dynamically reallocate budget between the persistent and retrieval tiers in response to the agent's recent experience, possibly using predictive or learned signals to anticipate which pages will be useful in the future rather than relying purely on historical frequency. A second direction is *multi-granularity memory*, where the agent maintains hierarchical state spanning turns, sessions, and entire task stages, enabling reasoning at the appropriate level of abstraction for each query. A third direction is *tighter integration with tool use and planning*: agents should not only remember information, but also strategically decide what to keep, retrieve, summarize, or recompute. Taken together, these directions suggest that long-horizon memory for agentic LLMs is still an early-stage research area where significant gains are achievable by jointly designing retention, retrieval, and reasoning strategies.

6 Related Work

Agent memory management. Recent LLM-agent systems increasingly treat memory as a managed resource rather than a passive conversation buffer. MemGPT [8] introduces an operating-system-inspired memory hierarchy to manage limited LLM context windows. MemoryBank [13] studies long-term user memory through memory storage, retrieval, updating, and forgetting. More recent work further moves toward agentic memory management: Agent Workflow Memory [10] stores reusable action workflows for future tool and web tasks; A-MEM [11] dynamically organizes memories through indexing, linking, and memory evolution; MemTool [4] focuses on short-term memory management for dynamic tool calling across multi-turn conversations; MEM1 [14] learns compact memory states for long-horizon agents; and Memory-as-Action [12] formulates memory editing as explicit agent actions. These works show that memory management is becoming central to long-horizon and tool-augmented agents, but many systems still rely on heuristic policies for deciding what to retain, retrieve, summarize, or discard.

Cache management. Cache management has long studied how to allocate limited fast storage under capacity constraints. Recent systems work continues to improve replacement policies beyond classical LRU/LFU-style designs. S3-FIFO⁴ shows that simple FIFO

queues can achieve strong cache-eviction performance across large-scale traces, while SIEVE⁵ further demonstrates that simple eviction mechanisms can be both efficient and scalable. Other recent work explores learned and adaptive cache management, including ML-assisted eviction, QoS-aware multi-tier cache replacement, learning-based page replacement, and dynamically resized adaptive caches. These studies provide useful principles for agent memory management: limited-capacity memory requires explicit policies for retention, eviction, prioritization, and adaptation. Unlike traditional caches, however, agent memories are semantic objects whose utility depends on future reasoning and tool execution, not only recency or frequency.

7 Conclusion

We proposed the FREQUENCY LOCALITY hypothesis and built PageContext, a two-tier memory management system for long-horizon agentic tasks, on top of it. We first verified the hypothesis empirically on *LoCoMo*, then designed PageContext to combine a persistent hot cache with retrieval-based context—maintaining task-wide information across queries while still admitting query-specific evidence. Our evaluation on *LoCoMo* across eight LLM backends shows that PageContext performs on par with strong retrieval-only baselines, with the persistent cache occasionally introducing noise that slightly hurts aggregate F1. The in-depth analysis attributes the gap to retrieval homogeneity, the granularity at which hot pages are defined, and the long-tail behaviour of token-overlap metrics under hallucinated outputs. Future work will explore adaptive and learned memory policies, multi-granularity memory representations, and heterogeneous retrieval strategies to further enhance long-horizon agentic reasoning.

8 Team Members Contribution

Two team members contribute equally to this project:

- Yunqi Li: Design the PageContext framework, conduct the experiments (on commercial LLMs), write the report (Section 1, 2, 4, and 6).
- Yuchen Wang: Design the PageContext framework, conduct the experiments (on open-source LLMs), write the report (Section 3, 5, and 7).

References

- [1] Alibaba Cloud. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [2] Deepseek. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638. doi:10.1038/s41586-025-09422-z
- [3] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.
- [4] Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, and James A. Burke. 2025. MemTool: Optimizing Short-Term Memory Management for Dynamic Tool Calling in LLM Agent Multi-Turn Conversations. arXiv:2507.21428 [cs.CL] <https://arxiv.org/abs/2507.21428>
- [5] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753* (2024).
- [6] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [7] OpenAI. 2026. OpenAI GPT-5 System Card. arXiv:2601.03267 [cs.CL] <https://arxiv.org/abs/2601.03267>

⁴<https://s3fifo.com/blog/>

⁵<https://cachemon.github.io/SIEVE-website/>

- [8] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] <https://arxiv.org/abs/2310.08560>
- [9] Kimi Team. 2026. Kimi K2.5: Visual Agentic Intelligence. arXiv:2602.02276 [cs.CL] <https://arxiv.org/abs/2602.02276>
- [10] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent Workflow Memory. arXiv:2409.07429 [cs.CL] <https://arxiv.org/abs/2409.07429>
- [11] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.CL] <https://arxiv.org/abs/2502.12110>
- [12] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. 2026. Memory as Action: Autonomous Context Curation for Long-Horizon Agentic Tasks. arXiv:2510.12635 [cs.AI] <https://arxiv.org/abs/2510.12635>
- [13] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. MemoryBank: Enhancing Large Language Models with Long-Term Memory. arXiv:2305.10250 [cs.CL] <https://arxiv.org/abs/2305.10250>
- [14] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents. arXiv:2506.15841 [cs.CL] <https://arxiv.org/abs/2506.15841>